# Rockchip USB Performance Analysis Guide

ID: RK-KF-YF-100

Release Version: V1.2.0

Release Date: 2026-06-22

Security Level: □Top-Secret □Secret □Internal ■Public

**DISCLAIMER**

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD. (“ROCKCHIP”) DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

**Trademark Statement**

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

**All rights reserved. ©2026. Rockchip Electronics Co., Ltd.**

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, Software Boulevard Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

## Preface

### Overview

This document provides methods for analyzing the transfer performance of USB 2.0/3.0 modules on the Rockchip platform, as well as the maximum supported transfer rates. The document introduces both USB Host and USB Device (Peripheral) parts. The USB Host section covers transfer performance analysis for USB Disk, USB Ethernet, and USB Camera. The USB Device (Peripheral) section covers transfer performance analysis for USB MTP, USB MSC (Mass Storage Class), USB Gadget Webcam, USB Rndis, and USB Gadget HID.

### Product Version

| Chip Models | Kernel Version |
| --- | --- |
| All Rockchip chips (except MCU) | Linux-3.10 and above |

### Intended Audience

This document (this guide) is primarily intended for the following engineers:

- Technical Support Engineers
- Software Development Engineers
- Hardware Engineers

### Revision History

| Date | Version | Author | Description |
| --- | --- | --- | --- |
| 2017-12-25 | V1.0 | Wu Liangfeng | Initial version |
| 2019-01-09 | V1.1 | Wu Liangfeng | Revised format using markdownlint |
| 2020-02-19 | V1.1.1 | Wu Liangfeng | Added disclaimer, trademark notice, and copyright notice |
| 2026-06-22 | V1.2.0 | Wu Liangfeng | Modified document format according to Rockchip documentation writing specifications<br>Corrected typos and technical errors<br>Added USB Disk speed optimization example<br>Added Webcam and HID descriptions based on Gadget configfs framework |

## Contents


Rockchip USB Performance Analysis Guide

- 1. USB Theoretical Transfer Rate Analysis
  - 1.1 USB Communication Flow
  - 1.2 USB Physical Layer Transfer Rate
  - 1.3 USB Protocol Layer Transfer Rate
  - 1.4 Main Factors Affecting USB Transfer Rate
- 2. USB Host Transfer Performance Analysis
  - 2.1 USB Disk Transfer Rate Analysis
  - 2.2 USB Ethernet Transfer Rate Analysis
  - 2.3 USB Camera Transfer Rate Analysis
- 3. USB Device Transfer Performance Analysis
  - 3.1 USB MTP Transfer Rate Analysis
  - 3.2 USB MSC Transfer Rate Analysis
  - 3.3 USB Gadget Webcam Transfer Rate Analysis
    - 3.3.1 USB Gadget Webcam [Legacy]
    - 3.3.2 USB Gadget Webcam [Configfs-UVC]
  - 3.4 USB Rndis Transfer Rate Analysis
  - 3.5 USB Gadget HID Transfer Rate Analysis
    - 3.5.1 USB Gadget HID [Legacy]
    - 3.5.2 USB Gadget HID [Configfs-HID]

# 1. USB Theoretical Transfer Rate Analysis

## 1.1 USB Communication Flow

The USB communication flow model is shown in Figure 1-1. It adopts a layered structure. The connection between a host and a USB device consists of connections at multiple layers. The USB bus interface layer provides the physical connection, sending and receiving data between the host and the device. The USB device layer is visible to the USB system software, which performs general USB operations on the device based on the device layer it observes. The USB application layer can provide additional functionality to the host through cooperating client software. Communication between the USB device layer and the application layer is logical; the actual physical communication corresponding to these logical communications is performed by the USB bus interface layer.

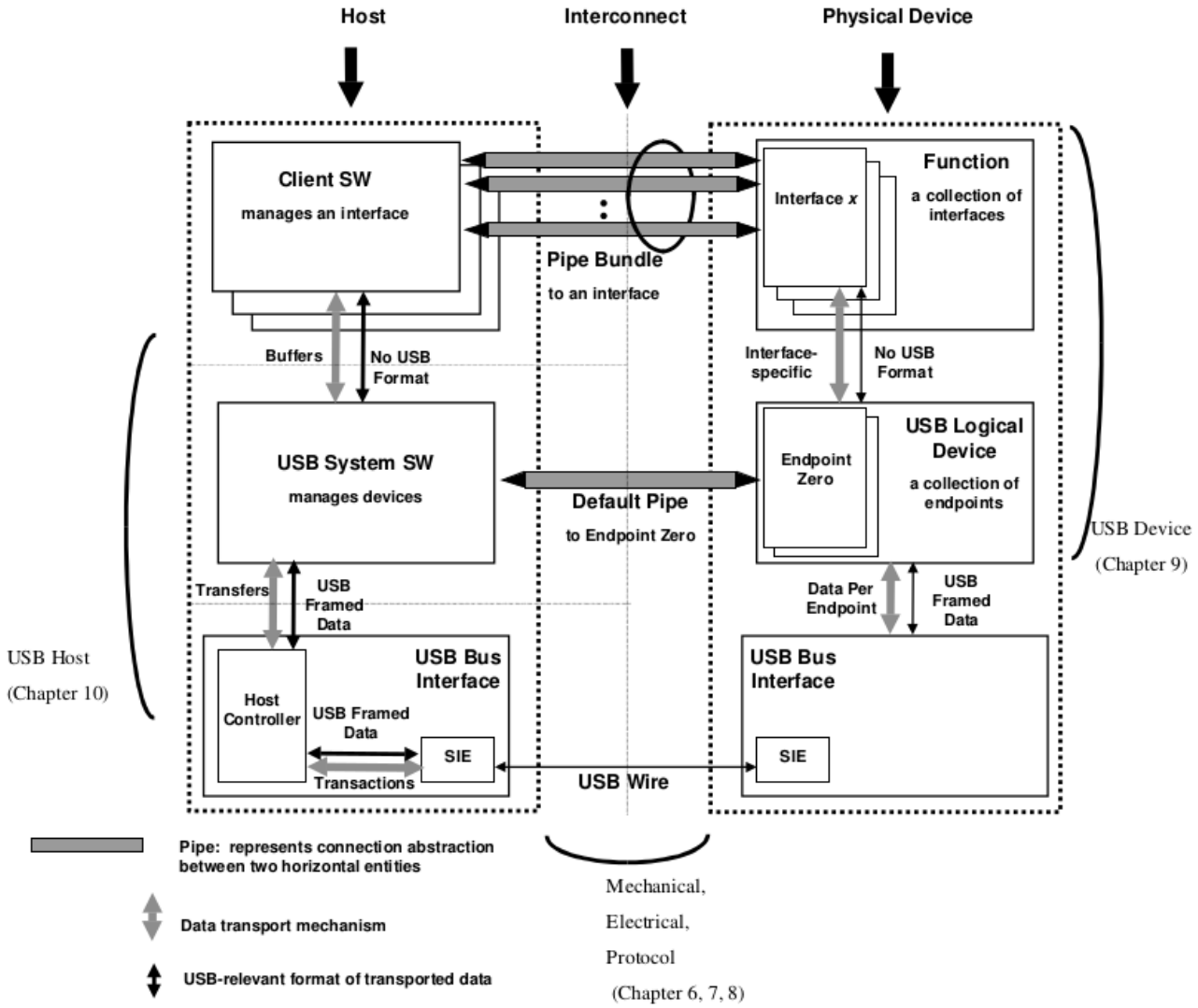



Figure 1-1 USB Communication Flow

## 1.2 USB Physical Layer Transfer Rate

The USB physical layer transfer rate refers to the transfer rate of the USB physical bus. The USB versions officially released by USB-IF and their physical layer transfer rates are shown in Table 1-1 below.

Table 1-1 USB Physical Layer Transfer Rate Support

| USB Version | Release Year | Max Speed | Rate Name | Encoding Scheme | Lane Count | Interface Type |
|---|---|---|---|---|---|---|
| USB 1.0 | 1996 | 1.5 Mbps | Low-Speed | NRZI | 1 | Type-A |
| USB 1.1 | 1998 | 12 Mbps | Full-Speed | NRZI | 1 | Type-A/B |
| USB 2.0 | 2000 | 480 Mbps | High-Speed | NRZI | 1 | Type-A/B/Mini/Micro |
| USB 3.0 / 3.2 Gen1x1 | 2008 | 5 Gbps | SuperSpeed | 8b/10b | 1 | Type-A/C |
| USB 3.1 Gen2 / 3.2 Gen2x1 | 2013 | 10 Gbps | SuperSpeed+ | 128b/132b | 1 | Type-A/C |
| USB 3.2 Gen1x2 | 2018 | 10 Gbps | SuperSpeed+ | 128b/132b | 2 | Type-C |
| USB 3.2 Gen2x2 | 2018 | 20 Gbps | SuperSpeed+ | 128b/132b | 2 | Type-C |
| USB4 Gen2x2 | 2019 | 20 Gbps | - | 128b/132b | 2 | Type-C |
| USB4 Gen3x2 | 2019 | 40 Gbps | - | 128b/132b | 2 | Type-C |
| USB4 v2 | 2022 | 80 Gbps | - | 128b/132b + PAM3 | 2 | Type-C |

**Note:**

1. USB4 v2 supports asymmetric configuration via PAM3 modulation, with downlink speeds up to 120 Gbps;
2. SuperSpeed and above are full-duplex, while USB 2.0 and below are half-duplex;
3. Actual effective throughput must deduct encoding overhead (approximately 20% for 8b/10b, approximately 3% for 128b/132b);
4. For the actual supported transfer rates of USB interfaces across the entire Rockchip platform, please refer to the document "Rockchip_Developer_Guide_USB_EN".

## 1.3 USB Protocol Layer Transfer Rate

Different USB device types use different USB device class protocols. For example: USB Disk uses a sub-specification of the USB Mass Storage Class specification (USB Mass Storage Class Bulk-Only Transport), USB Camera uses the USB Video Class specification, USB HID uses the USB Human Interface Devices Class specification. For more specifications, please visit the USB-IF official website.

Different USB device class protocols define different data packet formats. Some device class protocols also require command packets and handshake packets. Moreover, different USB device class protocols use different USB transfer types. For example: USB Disk uses Bulk Transfers, USB Camera uses Isochronous Transfers, and USB HID uses Interrupt Transfers.

The USB system supports four transfer types:

- Control Transfers

  Mainly used for enumerating devices when they are connected and for other device-specific operations;

- Interrupt Transfers

  Used for reliable transmission of small amounts of data with strict latency requirements, such as keyboards, gamepads, and other HID devices;

- Bulk Transfers

  Used for reliable transmission of large amounts of data with relaxed latency requirements, such as USB flash drives and USB Ethernet adapters;

- Isochronous Transfers

  Used for real-time data transmission where reliability is not critical, such as USB Camera and USB Audio;

Different USB transfer types have no difference on the USB physical bus. They differ only in transfer mechanism, host scheduling of transfer tasks, USB bandwidth occupancy limits, and maximum packet length, as shown in Table 1-2 below.

Table 1-2 Four USB Transfer Types

| Transfer Type | USB 2.0 | USB 3.0 |
| --- | --- | --- |
| Control Transfer | Max Packet: 64 Bytes<br>Burst: Not supported<br>Supports flow control only for OUT (PING) | Max Packet: 512 Bytes<br>Burst: 1<br>Supports flow control (ERDY) |
| Interrupt Transfer | Max Packet: 0 ~ 1024 × 3 Bytes<br>Theoretical effective bandwidth: 23.44 MBps<br>Actual achievable bandwidth: 23.44 MBps<br>Burst: Not supported<br>Flow control not supported | Max Packet: 0 ~ 1024 × 3 Bytes<br>Theoretical effective bandwidth: 23.44 MBps<br>Actual achievable bandwidth: 23.44 MBps<br>Burst: 1 ~ 3<br>Supports flow control (ERDY) |
| Bulk Transfer | Max Packet: 512 Bytes<br>Theoretical effective bandwidth: 53.25 MBps<br>Actual achievable bandwidth: ~40 MBps<br>Burst: Not supported<br>Supports flow control only for OUT (PING) | Max Packet: 1024 Bytes<br>Theoretical effective bandwidth: 500 MBps<br>Actual achievable bandwidth: ~400 MBps<br>Burst: 1 ~ 16<br>Supports flow control (ERDY) |
| Isochronous Transfer | Max Packet: 0 ~ 1024 × 3 Bytes<br>Theoretical effective bandwidth: 23.44 MBps<br>Actual achievable bandwidth: 23.44 MBps<br>Burst: Not supported<br>Flow control not supported | Max Packet: 0 ~ 1024 × 16 × 3 Bytes<br>Theoretical effective bandwidth: 375 MBps<br>Actual achievable bandwidth: ~300 MBps<br>Burst: 1 ~ 16<br>Flow control not supported |

**Note:**

1. BULK transfer is the transfer type with the highest bandwidth utilization in USB 3.0, with no fixed timing overhead, no reserved bandwidth, and transmission only when the bus is idle, with no latency guarantee;
2. The theoretical effective bandwidth of USB 3.0 has already deducted the 8b/10b encoding overhead, but does not include protocol layer overhead (packet headers, link layer management, ACK, etc.);
3. Actual bandwidth is limited by controller performance, data payload size (recommended ≥ 1MB), host scheduling, and other factors, typically achieving 75-80% of the theoretical value;
4. The bandwidth values for interrupt transfer and isochronous transfer in the table are calculated based on 1024 × 3 × 8 = 24576 Bytes/ms, converted at 1MB = 1024KB (approximately 23.44 MBps); the bulk transfer bandwidth value is calculated based on 512 × 13 × 8 = 53248 Bytes/ms, converted at 1MB = 1000KB (approximately 53.25 MBps);
5. USB 3.0 isochronous transfer does not support NRDY/ERDY flow control at the endpoint layer, but the link layer still has a Credit-based flow control mechanism to prevent receive buffer overflow.

## 1.4 Main Factors Affecting USB Transfer Rate

The USB transfer rate is mainly affected by the following aspects:

1. **USB Signal Quality**

   USB signal quality can be evaluated by testing the USB eye diagram. If the USB signal quality is poor, it can easily cause packet loss and retransmission on the USB physical bus, affecting the USB transfer rate.

2. **USB Controller AHB CLK and DMA Burst Length**

   Higher AHB CLK and larger DMA Burst Length can improve the DMA transfer efficiency of the USB controller.

3. **CPU Operating Frequency**

   The CPU operating frequency affects code execution efficiency and data copy (memcpy) efficiency. Therefore, increasing the CPU frequency can improve the execution efficiency of USB drivers and the application layer, thereby improving the USB transfer rate.

4. **URB (USB Request Block) Buffer Size Submitted by USB Device Class Driver**

   The submitted URB buffer size is the data block size requested for each transfer by the USB controller. For USB MSC (Mass Storage Class) devices, such as USB Disk and USB MTP, if the impact of data block size on the read/write performance of storage media (EMMC or NAND) is not considered, the buffer size can be increased as much as possible to reduce the number of interrupts during USB controller transfers. The Rockchip USB DWC2 controller supports a maximum transfer of 512KB at a time, and the USB DWC3 controller supports a maximum transfer of 16MB - 1B at a time.

5. **Storage Media Read/Write Performance**

   For USB MSC (Mass Storage Class) devices, such as USB Disk, the read/write rate is affected by the storage media on the USB Host side and the storage media of the USB flash drive itself.

6. **File System Format**

   USB flash drive copy rate is easily affected by the file system. Common file system formats include: VFAT, EXT4, and NTFS. For the transfer mechanism of VFAT/EXT4 file system formats, the kernel's block layer automatically merges small data blocks into 120K before writing to disk. However, NTFS write-to-disk operations are performed in user space, and data blocks are not merged by the block layer. Therefore, if the data block requested by the application layer each time is too small (e.g., 4KB), the copy rate of a USB flash drive with NTFS file system format will generally be significantly slower than that with VFAT/EXT4 file system format. For detailed analysis, please refer to USB Disk Transfer Rate Analysis

# 2. USB Host Transfer Performance Analysis

## 2.1 USB Disk Transfer Rate Analysis

**Test Method:**

Assume that after the USB Disk is inserted into the USB Host interface, the enumerated information is as follows:

The disk partition is:

`/dev/block/sda1`

The mount path is:

`/mnt/media_rw/0E64-5F76/`

USB Host EMMC file storage path: `/sdcard/.`

The following methods are explained based on the above assumptions.

**Note:**

- **The test file should be large enough**. For example, when testing on a 3399 EVB with 4G DDR, it is recommended that the test file be 2G to ensure that data is flushed from cache to disk. In actual testing, if a 1024MB file is used, the rate is inaccurate. The reason is that `/proc/sys/vm/dirty_background_ratio` is generally 5%, meaning that with 4G of memory, the file system cache dirty pages must reach 4G × 5% = 204.8MB before the system asynchronously flushes the cached dirty pages to the storage media.
- The dd command can be used with **conv=fsync** to write cached data to disk.

There are usually two methods for testing USB Disk transfer rate:

- **Method 1: Manually or using the cp command to copy large files (such as movies), then time it and calculate the rate;**

  step1. Use the dd command to create a test file for use in the copy test in step2

  `busybox dd if=/dev/zero of=/mnt/media_rw/0E64-5F76/test bs=64K count=32K`

  step2. Use the cp command to count the copy time

  `echo 3 > /proc/sys/vm/drop_caches` `time cp /mnt/media_rw/0E64-5F76/test /sdcard/.`

  The above method tests the USB Disk read rate. If you need to test the USB Disk write rate, simply modify the read/write paths.

- **Method 2: Use the dd command to test**

  **Method for testing USB Disk read rate:**

  step1. Use the dd command to create a test file for subsequent copy testing

  `busybox dd if=/dev/zero of=/mnt/media_rw/0E64-5F76/test bs=64K count=32K`

step2.

Clear the cache before each dd command execution

`echo 3 > /proc/sys/vm/drop_caches`

Test the read rate with 4K data blocks

`busybox dd if=/mnt/media_rw/0E64-5F76/test of=/dev/null bs=4K count=256K`

step3.

Clear the cache before each dd command execution

`echo 3 > /proc/sys/vm/drop_caches`

Test the read rate with 64K data blocks

`busybox dd if=/mnt/media_rw/0E64-5F76/test of=/dev/null bs=64K count=16K`

step4.

Clear the cache before each dd command execution

`echo 3 > /proc/sys/vm/drop_caches`

Test the read rate with 128K data blocks

`busybox dd if=/mnt/media_rw/0E64-5F76/test of=/dev/null bs=128K count=8K`

step5.

Clear the cache before each dd command execution

`echo 3 > /proc/sys/vm/drop_caches`

Test the read rate with 512K data blocks

`busybox dd if=/mnt/media_rw/0E64-5F76/test of=/dev/null bs=512K count=2K`

**Method for testing USB Disk write rate:**

step1.

Clear the cache before each dd command execution

`echo 3 > /proc/sys/vm/drop_caches`

Test the write rate with 4K data blocks

`busybox dd if=/dev/zero of=/mnt/media_rw/0E64-5F76/test bs=4K count=256K`

step2.

`rm /mnt/media_rw/0E64-5F76/test`

`sync`

`echo 3 > /proc/sys/vm/drop_caches`

Test the write rate with 64K data blocks

`busybox dd if=/dev/zero of=/mnt/media_rw/0E64-5F76/test bs=64K count=16K`

step3.

`rm /mnt/media_rw/0E64-5F76/test`

`sync`

`echo 3 > /proc/sys/vm/drop_caches`

Test the write rate with 128K data blocks

```
busybox dd if=/dev/zero of=/mnt/media_rw/0E64-5F76/test bs=128K count=8K
```

step4.

```
rm /mnt/media_rw/0E64-5F76/test
```

```
sync
```

```
echo 3 > /proc/sys/vm/drop_caches
```

Test the write rate with 512K data blocks

```
busybox dd if=/dev/zero of=/mnt/media_rw/0E64-5F76/test bs=512K count=2K
```

**Note:**

- Method 1 can measure the actual data read/write rate of USB Disk, but the test results are simultaneously affected by the Host-side storage media, file system performance, and USB controller transfer rate, and cannot accurately describe the transfer performance of the USB controller.
- Method 2 can eliminate the influence of Host-side storage media and file system, and accurately measure the transfer performance of the USB controller reading and writing to the USB flash drive. However, it should be noted that the file system of the USB Disk (Device side) will still affect the copy. To eliminate the influence of the USB Disk file system, you need to directly read/write the sd* partition node under the /dev/ path, but this will also destroy the original data on the USB Disk, so use with caution.
- The dd command can be used with **conv=fsync** to write cached data to disk, which can completely eliminate the impact of cache. Of course, if the test file is large enough, the impact of cache is also relatively small.
- When testing, it is recommended to use both methods. First use Method 1 for testing. If the test results cannot achieve the expected transfer rate, then further use Method 2 to analyze whether the transfer bottleneck is in the USB controller. Detailed analysis methods will be explained in the "Test Result Analysis" section.

**Test Results:**

- **USB 2.0 Host**

  Using the dd command for testing, the USB Disk read/write rate of the Rockchip USB 2.0 Host interface is typically **25 MBps ~ 35 MBps**.

- **USB 3.0 Host**

  Using the dd command for testing, the read/write rate of USB 3.0 Disk (without UAS support) on the Rockchip USB 3.0 Host interface is typically **60 MBps ~ 100 MBps**. For USB 3.0 Disks that support **UAS**, the maximum read/write rate is approximately **350 MBps**.

**Note:**

- Different USB 3.0 Disks may have significantly different read/write performance, and some USB 3.0 Disks have read rates 3-5 times higher than write rates. Therefore, when evaluating the transfer rate of a USB controller, you should test as many different models of USB 3.0 Disks as possible, and use the same test method to test the read/write rate of the same USB 3.0 Disk on PC Ubuntu for comparison reference.
- UAS support: RK3399/RK3328 USB 3.0 controllers support UAS (USB Attached SCSI Protocol), which can greatly improve the read/write rate of USB Disk. To use the UAS function, first, the kernel needs to

enable CONFIG_USB_UAS (enabled by default in both kernel 3.10 and kernel 4.4), and second, the USB 3.0 Disk needs to support the UAS function.

- You can confirm whether the UAS protocol is used from the enumeration log

```
[80373.513866] usb 4-1: new SuperSpeed USB device number 2 using xhci_hcd
[80373.534854] usb 4-1: New USB device found, idVendor=174c, idProduct=55aa
[80373.534871] usb 4-1: New USB device strings: Mfr=2, Product=3, SerialNumber=1
[80373.534873] usb 4-1: Product: CHIPFANCIER
[80373.534874] usb 4-1: Manufacturer: WINTOGO
[80373.534876] usb 4-1: SerialNumber: 000001485761816
[80373.537036] scsi host6: uas (indicates communication using UAS protocol)
[80373.537648] scsi 6:0:0:0: Direct-Access    WINTOGO  CHIPFANCIER      0    PQ: 0 ANSI: 6
```

**Test Result Analysis and Performance Optimization Directions:**

**Example:**

This document uses the typical application scenario of RK3399 EVB reading/writing USB 3.0 Disk for analysis. Other application scenarios can refer to the following methods.

Test environment: RK3399 EVB (2G DDR + 16G EMMC)

USB 3.0 Disk model: SanDisk Type-C USB 3.0 32G (NTFS file system)

Test scenario: Test the rate of copying large files from USB 3.0 Disk to 3399 EMMC

Test commands: cp, dd

Assume USB 3.0 Disk mount path: /mnt/media_rw/0E64-5F76/

EMMC file storage path: /sdcard/.

**Note:**

If the test machine has 4G DDR, it is recommended that the test file be **2GB** to avoid cache effects. In addition, the dd command can be used with **conv=fsync** to write cached data to disk, which can completely eliminate the impact of cache. Of course, if the test file is large enough, the impact of cache is also relatively small.

**step1. Use the dd command to create a test file (1GB size) for subsequent copy testing**

`busybox dd if=/dev/zero of=/mnt/media_rw/0E64-5F76/test bs=64K count=16K`

**step2. Use the cp command to test the rate of copying large files from USB 3.0 Disk to 3399 EMMC**

`time cp /mnt/media_rw/0E64-5F76/test /sdcard/.` (test is a 1GB file)

Test results: 0m48.42s real 0m00.29s user 0m18.17s system

Copy rate: 1024M/48.42s ≈ 21MBps This rate is significantly slower than the normal USB 3.0 copy rate

**step3. Copy directly from /dev/block/USB Disk partition to EMMC to troubleshoot the impact of 3399 file system**

`time cp /dev/block/sda1 /sdcard/test` (This command can bypass the file system and copy directly)

Copy rate is approximately: 20 MBps

This indicates that the slow copy rate is not related to the 3399 file system itself.

**step4. Use the dd command to test the read/write performance of USB flash drive and 3399 EMMC separately**

- **Test USB 3.0 Disk read performance**

  Clear the cache before each dd command execution

  `echo 3 > /proc/sys/vm/drop_caches`

  Test the read rate with 4K data blocks

  `busybox dd if=/mnt/media_rw/0E64-5F76/test of=/dev/null bs=4K count=256K`

  Result: 1073741824 bytes (1.0GB) copied, 14.183164 seconds, 72.2MB/s

  Clear the cache before each dd command execution

  `echo 3 > /proc/sys/vm/drop_caches`

  Test the read rate with 64K data blocks

  `busybox dd if=/mnt/media_rw/0E64-5F76/test of=/dev/null bs=64K count=16K`

  Result: 1073741824 bytes (1.0GB) copied, 12.546069 seconds, 81.6MB/s

  Clear the cache before each dd command execution

  `echo 3 > /proc/sys/vm/drop_caches`

  Test the read rate with 128K data blocks

  `busybox dd if=/mnt/media_rw/0E64-5F76/test of=/dev/null bs=128K count=8K`

  Result: 1073741824 bytes (1.0GB) copied, 12.601817 seconds, 81.3MB/s

  Clear the cache before each dd command execution

  `echo 3 > /proc/sys/vm/drop_caches`

  Test the read rate with 512K data blocks

  `busybox dd if=/mnt/media_rw/0E64-5F76/test of=/dev/null bs=512K count=2K`

  Result: 1073741824 bytes (1.0GB) copied, 12.402147 seconds, 82.6MB/s

- **Test 3399 EMMC write performance**

  Clear the cache before each dd command execution

  `echo 3 > /proc/sys/vm/drop_caches`

  Test the write rate with 4K data blocks

  `busybox dd if=/dev/zero of=/sdcard/test bs=4K count=256K`

  Result: 1073741824 bytes (1.0GB) copied, 39.525297 seconds, 25.9MB/s

  `rm /sdcard/test`

  `sync`

  `echo 3 > /proc/sys/vm/drop_caches`

  Test the write rate with 64K data blocks

  `busybox dd if=/dev/zero of=/sdcard/test bs=64K count=16K`

  Result: 1073741824 bytes (1.0GB) copied, 37.505206 seconds, 27.3MB/s

`rm /sdcard/test`

`sync`

`echo 3 > /proc/sys/vm/drop_caches`

Test the write rate with 128K data blocks

`busybox dd if=/dev/zero of=/sdcard/test bs=128K count=8K`

Result: 1073741824 bytes (1.0GB) copied, 37.461603 seconds, 27.3MB/s

`rm /sdcard/test`

`sync`

`echo 3 > /proc/sys/vm/drop_caches`

Test the write rate with 512K data blocks

`busybox dd if=/dev/zero of=/sdcard/test bs=512K count=2K`

Result: 1073741824 bytes (1.0GB) copied, 37.086531 seconds, 27.6MB/s

From the above test results, the read performance of USB 3.0 Disk is very good, with small block and large block read performance basically consistent. However, the write performance of EMMC is obviously poor. The transfer rate bottleneck for copying large files from USB 3.0 Disk to 3399 EMMC is in the 3399's EMMC chip.

The EMMC chip analyzed above has slow write speeds for 4K, 64K, 128K, and 512K data blocks. However, on the Rockchip platform, most EMMC chips may only have slow 4K write rates (including USB 3.0 Disk as well). Therefore, if the tested EMMC write rate is only slow for 4K, further analysis is needed to determine whether it is related to 4K.

**step5. Analyze the behavior of cp command and block layer**

Analysis tools: **strace**, **blktrace**

- **Use strace command to trace cp behavior**

  `strace cp /mnt/media_rw/0E64-5F76/test /sdcard/.`

  Results show: On the RK3399 platform, the cp command reads and writes data blocks of 4KB

- **Use blktrace command to analyze block layer merge behavior**

  Several important parameters related to copying:

  `max_hw_sectors_kb`

  `max_sectors_kb`

  max_hw_sectors_kb and max_sectors_kb control the data block size written to storage media after the block layer performs merge operations. For USB flash drives, it is generally 120KB, and for EMMC, it is generally 512KB.

  EMMC:

  ```
  /sys/block/mmcblk1/queue/max_hw_sectors_kb
  /sys/block/mmcblk1/queue/max_sectors_kb
  ```

  USB Disk:

```
/sys/block/sda/queue/max_hw_sectors_kb
/sys/block/sda/queue/max_sectors_kb
```

Among them, the max_hw_sectors_kb of USB Disk is set in the code:

```
drivers/usb/storage/scsiglue.c
/*limit the total size of a transfer to 120 KB */
.max_sectors =  240, (unit is block, i.e., 512 bytes)
```

This parameter can be configured by the upper layer, such as RK3399 Type-C1 port:

`/sys/devices/platform/usb@fe900000/fe900000.dwc3/xhci-hcd.9.auto/usb4/4-1/4-1:1.0/host0/target0:0:0/0:0:0:0/max_sectors`

View IO scheduler:

`rk3399:/data# cat /sys/block/mmcblk1/queue/scheduler`

`noop [cfq]` (Using cfq indicates that merge behavior will theoretically be performed to merge small data blocks)

blktrace is a built-in Android tool, but it may not be built by default. You need to modify the mk file in the blktrace folder, compile the tool, and then use adb push to /system/bin.

Modify the Android.mk file: Change the first line of the file from `BUILD_BLKTRACE:= false` to `BUILD_BLKTRACE:= true`

To use blktrace, the kernel needs to add CONFIG_BLK_DEV_IO_TRACE, as follows:

```
Kernel hacking  --->
    [*]Tracers  --->
        [*]  Support for tracing block IO actions
```

For blktrace usage methods, refer to:

**Capture blktrace data:**

1. Create a folder to save data

   `mkdir /dev/blktrace`

2. Start blktrace to capture data

   `blktrace -d /dev/block/mmcxxx -o /dev/blktrace/blktrace &`

   (Where /dev/block/mmcxxx is the block to be analyzed, such as 3399's mmcblk1)

3. Execute the copy command

   `busybox dd if=/dev/zero of=/data/test bs=4K count=256K` (Execute copy)

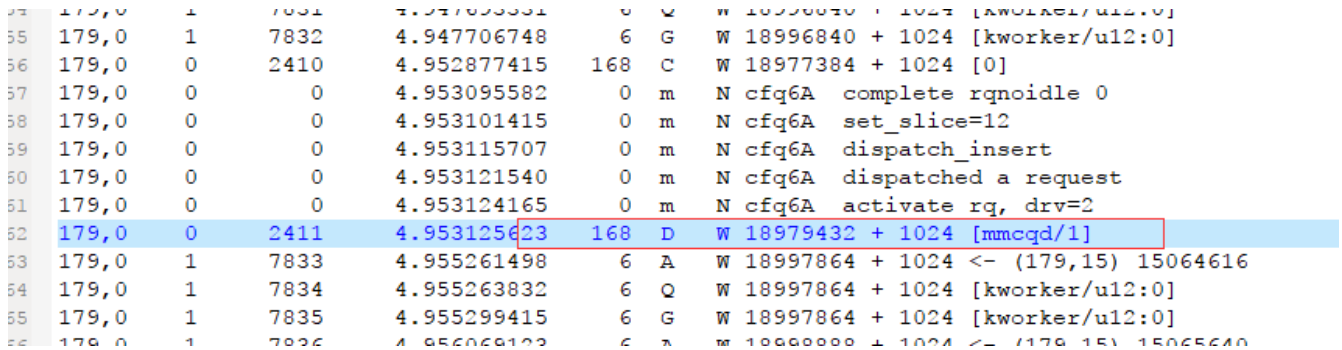
4. Stop blktrace

   After copying is complete, kill the blktrace thread, and the blktrace data will be automatically saved in the /dev/blktrace path. You can also add the -w parameter after blktrace to indicate that blktrace will automatically shut down after tracing for a specified number of seconds.

**Analyze blktrace data:**

`blkparse -i /dev/blktrace/blktrace.blktrace.0 -o /dev/blktrace/blkparse.txt`

According to blktrace statistics, cfq merges 4K small block data into 1024 sectors, i.e., 512K, before writing to EMMC, as shown in Figure 2-1 below.



```
179,0    1     7832     4.947706748     6  G   W 18996840 + 1024 [kworker/u12:0]
179,0    0     2410     4.952877415   168  C   W 18977384 + 1024 [0]
179,0    0        0     4.953095582     0  m   N cfq6A  complete rqnoidle 0
179,0    0        0     4.953101415     0  m   N cfq6A  set_slice=12
179,0    0        0     4.953115707     0  m   N cfq6A  dispatch_insert
179,0    0        0     4.953121540     0  m   N cfq6A  dispatched a request
179,0    0        0     4.953124165     0  m   N cfq6A  activate rq, drv=2
179,0    0     2411     4.953125623   168  D   W 18979432 + 1024 [mmcqd/1]
179,0    1     7833     4.955261498     6  A   W 18997864 + 1024 <- (179,15) 15064616
179,0    1     7834     4.955263832     6  Q   W 18997864 + 1024 [kworker/u12:0]
179,0    1     7835     4.955299415     6  G   W 18997864 + 1024 [kworker/u12:0]
```

Figure 2-1 blktrace data

This indicates that the merge behavior of the 3399's block layer is functioning correctly.

For more detailed blktrace usage methods, please refer to blktrace IO analysis

**step6. Analyze the impact of USB Disk file system format**

Common file system formats for USB Disk include: VFAT, EXT4, and NTFS. For the transfer mechanism of VFAT/EXT4 file system formats, the kernel's block layer automatically merges small data blocks into large data blocks before writing to disk. However, NTFS is based on FUSE, and write-to-disk operations are performed in user space with additional copy operations. Therefore, if the data block requested by the application layer each time is too small (e.g., 4KB), for USB Disk with NTFS file system format, the copy rate may be lower than that with VFAT/EXT4 file system format when the CPU is frequency-scaling. For example, on the RK3399 Android platform, the cp command requests only 4KB data blocks each time, so on the RK3399 Android platform, the read/write rate of NTFS-formatted USB 3.0 Disk tends to be relatively poor.

**Optimization Directions:**

Based on the above analysis, the main factors affecting USB Disk copying are as follows:

- USB Disk read/write performance;
- Storage media read/write performance on the USB Host side;
- USB Host controller transfer performance;
- File system format;
- USB data block size per request;
- CPU operating frequency.

You can refer to the methods provided in this document to first locate the transfer bottleneck, then perform optimization.

Taking the RK3562 platform USB flash drive transfer rate optimization as an example:

1. Disable RK3562 CPU IDLE to avoid slow USB interrupt processing caused by long CPU exit idle time.

```
cpunum=$(cat /proc/cpuinfo | grep processor | wc -l)
for i in `seq 0 $(($cpunum-1))`
do
echo 1 > /sys/devices/system/cpu/cpu$i/cpuidle/state0/disable
echo 1 > /sys/devices/system/cpu/cpu$i/cpuidle/state1/disable
done
```

2. Set RK3562 CPU to high frequency to resolve the issue of unstable USB transfer rate under high system load scenarios.

   Modify the CPU scheduling policy to 'performance'. This mode keeps the CPU operating at the highest frequency at all times to achieve maximum performance.

   ```
   cpunum=$(cat /proc/cpuinfo | grep processor | wc -l)
   for i in `seq 0 $(($cpunum-1))`
   do
   echo performance > /sys/devices/system/cpu/cpu$i/cpufreq/scaling_governor
   done
   ```

3. Increase max_sectors, which is equivalent to making the block size of each USB flash drive transfer larger, reducing the number of times it is preempted by other USB devices.

   First, determine the USB bus where the USB flash drive is mounted through the kernel USB flash drive enumeration information, for example: usb 1-1.3: Manufacturer: Kingston, then find the path to the max_sectors parameter of the USB flash drive

   `/sys/devices/platform/fed00000.usb/usb1/1-1/1-1.3/1-1.3:1.0/host0/target0:0:0/0:0:0:0/max_sectors`

   Set the max_sectors parameter value to 512 or 1024:

   `echo 512 > /sys/devices/platform/fed00000.usb/usb1/1-1/1-1.3/1-1.3:1.0/host0/target0:0:0/0:0:0:0/max_sectors`

## 2.2 USB Ethernet Transfer Rate Analysis

**Test Method:**

Test tool: iperf

**The iperf command** is a network performance testing tool. iperf can test TCP and UDP bandwidth quality. iperf can measure maximum TCP bandwidth with various parameters and UDP characteristics. iperf can report bandwidth, latency jitter, and packet loss.

- iperf official website:

  https://iperf.fr/

- iperf download address:

  Supports Windows/Android/Ubuntu/macOS and other systems, download address

  https://iperf.fr/iperf-download.php

  In addition, Ubuntu systems can directly install iperf using the following command

  `sudo apt install iperf`

  Android systems can install iperf.apk

- iperf usage:

  https://iperf.fr/iperf-doc.php

Bandwidth testing usually adopts **UDP mode** because it can measure the limit bandwidth, latency jitter, and packet loss rate. When testing, first use the link's theoretical bandwidth as the data sending rate for testing. For example, if the theoretical bandwidth of the link from the client to the server is 100Mbps, first test with `-b 100M`, then based on the test results (including actual bandwidth, latency jitter, and packet loss rate), test again with the actual bandwidth as the data sending rate. You will find that the latency jitter and packet loss rate are much better than the first time. Repeat the test several times to obtain a stable actual bandwidth.

**Example**

**UDP Mode**

PC Ubuntu serves as the server, and RK3399's USB 2.0/USB 3.0 Host interface connects to a UGREEN USB 3.0 Ethernet, then connects to the PC's Ethernet port via an Ethernet cable, forming a complete test link.

PC Ubuntu server side:

First, configure the IPv4 address, subnet mask, and gateway, as shown in Figure 2-2




Figure 2-2 PC Ubuntu IP Address Configuration

Then, execute the following command:

```
wlf@wlf-ubuntu:~$ iperf -u -s
------------------------------------------------------------
Server listening on UDP port 5001
Receiving 1470 byte datagrams
UDP buffer size:  208 KByte (default)
------------------------------------------------------------
```

3399 client side:

First, view the USB Ethernet adapter information

```
eth1      Link encap:Ethernet  HWaddr 60:38:E0:E3:20:96
          inet6 addr: fe80::6238:e0ff:fee3:2096/64 Scope: Link
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:2 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 TX bytes:168
```

Then, configure the IP address

`ifconfig eth1 192.168.1.2 up`

Then execute the following command to start bandwidth testing

```
iperf -u -c 192.168.1.1 -b 1000M -t 60
```

**Note:** -b indicates the bandwidth used in UDP mode, in bits/sec. For USB 2.0 Ethernet adapters, it is recommended to set -b 100M, and for USB 3.0 Ethernet adapters, it is recommended to set -b 1000M.

**Test Results:**

Using UGREEN USB 3.0 Ethernet, the actual test bandwidth is as follows:

```
------------------------------------------------------------
Client connecting to 192.168.1.1, UDP port 5001
Sending 1470 byte datagrams
UDP buffer size:  208 KByte (default)
------------------------------------------------------------
[  3] local 192.168.1.2 port 40116 connected with 192.168.1.1 port 5001
[ ID] Interval       Transfer     Bandwidth
[  3]  0.0-60.0 sec  5.61 GBytes   803 Mbits/sec
[  3] Sent 4096616 datagrams
```

Using the same test method, the bandwidth of CISCO USB 2.0 Ethernet is approximately 95 Mbps

**Test Result Analysis:**

The normal transfer rates of USB Ethernet on the Rockchip platform are as follows:

USB 2.0 Host: Both uplink and downlink are **95 ~ 100 Mbps**

USB 3.0 Host: Both uplink and downlink are **750 ~ 800 Mbps**

If the actual product test cannot achieve the above rates, please first use ifconfig and tcpdump tools to check whether the bit error rate is too high. If the bit error rate is too high, please test the USB eye diagram to confirm whether the USB signal quality meets the requirements.

## 2.3 USB Camera Transfer Rate Analysis

**Test Method:**

Use the system's built-in Camera APK for image preview, and execute logcat via serial port to view the printed real-time frame rate, as follows:

`CameraHal: debugShowFPS(622): Camera 1312 Frames, 30.000 FPS`

uvc driver debugging methods:

Print uvc trace information

```
echo 0xffff > /sys/module/uvcvideo/parameters/trace
echo 8 > /proc/sysrq-trigger
```

Disable uvc trace

`echo 0 > /sys/module/uvcvideo/parameters/trace`

View uvc decode statistics

`cat /d/usb/uvcvideo/*/stats`

**Test Results:**

The maximum frame rates supported by USB Camera on the Rockchip platform are as follows:

Table 2-1 Rockchip Platform USB Camera Maximum Frame Rate

| USB Version | YUYV Format | MJPEG Format | H264 Format |
|---|---|---|---|
| USB 2.0 | 640 × 480 @30fps | 1280 × 720 @30fps | 1920 × 1080 @30fps |
| USB 3.0 | 640 × 480 @60fps | 1280 × 720 @60fps | 1920 × 1080 @60fps |

**Note:** The prerequisite for the above frame rate testing is that only one USB Camera is connected to each USB controller. If two or more USB Cameras are connected simultaneously, the actual measured values shall prevail.

**Test Result Analysis and Performance Optimization:**

Most USB Cameras use isochronous transfer type for USB communication. The theoretical maximum bandwidth of USB 2.0 isochronous transfer is 23.44 MBps, and the theoretical maximum bandwidth of USB 3.0 isochronous transfer is 375 MBps. The maximum frame rate of USB Camera is generally affected by the following two aspects:

a). The maximum frame rate that the USB Camera can actually output;

b). The isochronous transfer performance of the USB Host driver;

For factor a), the performance of the USB Camera can be verified on a PC. For factor b), consider improving the USB QOS priority, increasing the USB interrupt response speed, and increasing the AHB CLK and DMA burst length of the USB controller.

# 3. USB Device Transfer Performance Analysis

## 3.1 USB MTP Transfer Rate Analysis

**Test Method:**

Connect the device under test to a PC, set USB to MTP mode, then manually copy large files (such as movies larger than 1GB), time it with a stopwatch, and calculate the average transfer rate.

Note the following during testing:

- The device under test should remain on the static desktop to avoid entering sleep mode;
- After connecting the device under test to the PC, wait for the media library scan to complete (can be observed through logcat output) before performing the copy operation;

**Test Results:**

**Example**

Test platform: RK3399 BOX SDK Android N + Kernel 4.4

Test description:

Read: RK3399 -> PC

Write: PC -> RK3399

16K, 64K, 1M represent the Tx and Rx Buffer Length of the MTP driver

During testing, both big and little cores of RK3399 are locked at maximum frequency

Table 3-1 RK3399 BOX SDK USB MTP Transfer Rate

| USB Version | Read 16K | Write 16K | Read 64K | Write 64K | Read 1M | Write 1M |
|---|---|---|---|---|---|---|
| USB 2.0 | 24.7 MBps | 22.8 MBps | 22.8 MBps | 21.5 MBps | 24.7 MBps | 25.6 MBps |
| USB 3.0 | 65.6 MBps | 109.0 MBps | 71.3 MBps | 107.5 MBps | 87.3 MBps | 110.0 MBps |

Test platform: RK3126C device Android O + Kernel 4.4

Table 3-2 RK3126C Device USB MTP Transfer Rate

| USB Version | Read 16K | Write 16K | Read 64K | Write 64K |
|---|---|---|---|---|
| USB 2.0 | 14.85 MBps | 7.1 MBps | 15.15 MBps | 7.8 MBps |

**Test Result Analysis and Performance Optimization Directions:**

USB MTP uses bulk transfer type. Theoretically, the maximum bandwidth of USB 2.0 bulk transfer is 53.25 MBps (13 transactions per micro-frame, i.e., 13 × 512 Bytes), and the maximum bandwidth of USB 3.0 bulk transfer is 500 MBps. However, the actual bulk transfer rate of the Rockchip platform's USB 2.0 controller is approximately 40 MBps, and the actual bulk transfer rate of the USB 3.0 controller is approximately 400 MBps. The actual MTP transfer is affected by the following factors, so the actual rate will be significantly lower than the theoretical rate.

- CPU operating frequency

  When the CPU is locked at high frequency, the MTP copy rate is significantly higher than when the CPU is frequency-scaling.

- Storage media read/write performance

  If the storage media read/write performance is poor, it will seriously affect the MTP transfer rate. For methods to evaluate storage media read/write performance, please refer to USB Disk Transfer Rate Analysis

  Kernel 4.4 provides a debugging interface to print the time consumed for each read/write to storage media (vfs_read/vfs_write) during MTP transfer. Usage is as follows:

  Query command:

  `cat /sys/kernel/debug/usb_mtp/status`

  Command to reset statistical state:

  `echo 0 > /sys/kernel/debug/usb_mtp/status`

- USB controller transfer performance

  Increasing the AHB CLK of the USB 2.0 controller to 150MHz and setting the DMA Burst to maximum can improve the DMA transfer performance of the USB controller, thereby improving the USB controller transfer rate.

- MTP driver Tx/Rx buffer size

  The USB 2.0 controller (DWC2) supports a maximum transfer of 512KB of data at a time, and the USB 3.0 controller (DWC3) supports a maximum transfer of 16MB - 1B. Therefore, the Tx/Rx buffer size of the MTP driver can be increased as much as possible (currently the MTP driver default is set to 64KB) to reduce the number of USB transfer interrupts. It is recommended to set the Tx/Rx buffer to 64KB for USB 2.0 controllers and 1MB for USB 3.0 controllers.

  The Kernel 4.4 MTP driver supports configuring Tx/Rx buffer through module parameters. The interfaces are as follows:

  `/sys/module/usb_f_mtp/parameters/mtp_tx_req_len` (Tx Buffer Length, unit: byte)

  `/sys/module/usb_f_mtp/parameters/mtp_tx_reqs` (Tx Buffer count)

  `/sys/module/usb_f_mtp/parameters/mtp_rx_req_len` (Rx Buffer Length, unit: byte)

## 3.2 USB MSC Transfer Rate Analysis

**Test Method:**

Using Linux-4.4 RK3399 EVB USB 3.0 MSC as the analysis example

Configure Mass storage as a module

```
<*>   USB Gadget Support  --->
      <M>   USB Gadget Drivers
      <M>     Mass Storage Gadget
```

Add the following patch to the Kernel:

```
index f3c7f15..5418d06 100644
--- a/arch/arm64/boot/dts/rockchip/rk3399.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3399.dtsi
@@ -404,7 +404,7 @@
                        compatible = "snps,dwc3";
                        reg = <0x0 0xfe800000 0x0 0x100000>;
                        interrupts = <GIC_SPI 105 IRQ_TYPE_LEVEL_HIGH 0>;
-                       dr_mode = "otg";
+                       dr_mode = "peripheral";
                        phys = <&u2phy0_otg>, <&tcphy0_usb3>;
                        phy-names = "usb2-phy", "usb3-phy";
                        phy_type = "utmi_wide";
```

Compile the kernel

Compile the Mass storage module

`make ARCH=arm64 modules M=drivers/usb/gadget`

Generate the following .ko files

`libcomposite.ko`

`usb_f_mass_storage.ko`

`g_mass_storage.ko`

After the system boots, first use the dd command to create a lun0.img in the data directory, then load the .ko files

```
insmod libcomposite.ko
insmod usb_f_mass_storage.ko
insmod g_mass_storage.ko  file=/data/lun0.img removable=1
```

Connect to PC Ubuntu, format lun0, and then Mass storage can be accessed normally from the PC

Use the following script to test the read rate

```
while true;
do time dd iflag=direct,nonblock if=/media/wlf/4625f014-95f3-419c-96c9-a38f91186eb2/test0
of=/dev/null bs=1024k count=400;
done
```

You can also use a USB 3.0 Agilent analyzer to test the transfer rate, as shown in Figure 3-1

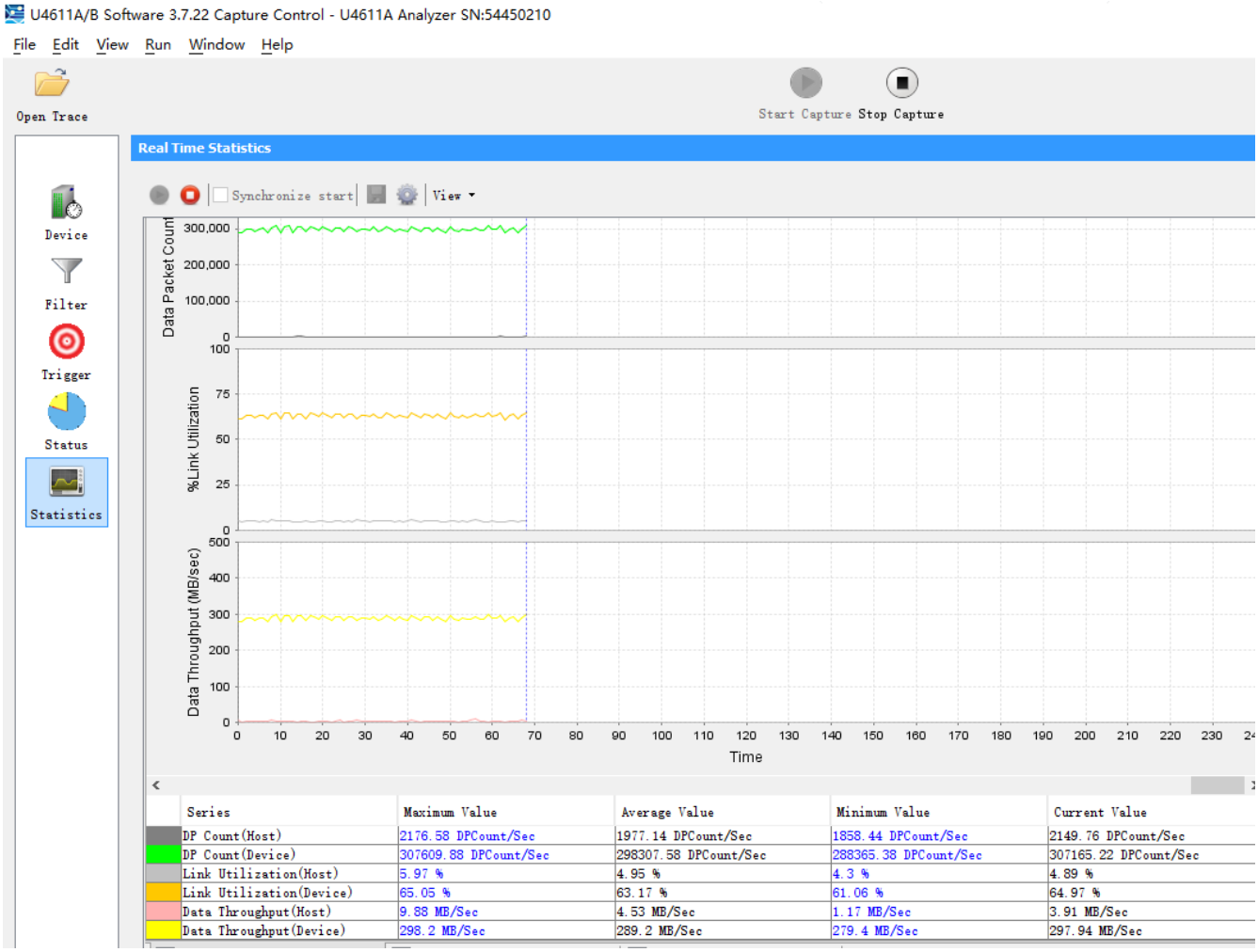



Figure 3-1 USB 3.0 Agilent Analyzer Transfer Rate Test

**Test Results and Analysis:**

RK3399 frequency-scaling: Read rate 250 MBps

RK3399 disable little cores, lock big cores at 1.8GHz: Read rate 300 MBps

```
3399 disable little cores
echo 0 > /sys/devices/system/cpu/cpu0/online
echo 0 > /sys/devices/system/cpu/cpu1/online
echo 0 > /sys/devices/system/cpu/cpu2/online
echo 0 > /sys/devices/system/cpu/cpu3/online

3399 lock big core frequency
echo userspace > /sys/devices/system/cpu/cpu4/cpufreq/scaling_governor
echo 1800000 >  /sys/devices/system/cpu/cpu4/cpufreq/scaling_setspeed
```

Conclusion: The USB MSC transfer rate is significantly affected by the CPU operating frequency. The higher the CPU frequency, the higher the read rate.

## 3.3 USB Gadget Webcam Transfer Rate Analysis

**Test Method:**

Test tools:

PC Windows: AMCAP software

PC Ubuntu: guvcview software (Installation method: sudo apt install guvcview)

Both AMCAP and guvcview software support real-time frame rate statistics and display.

### 3.3.1 USB Gadget Webcam [Legacy]

To support USB Gadget Webcam, in addition to enabling the UVC gadget driver in the kernel, the application layer also needs to add UVC processing. Currently, both Kernel-3.10 and Kernel-4.4 on the Rockchip platform support the USB gadget driver. At the application layer, only the RV1108 SDK supports UVC gadget. For other chip platforms, you can refer to the RV1108 SDK or the usb gadget uvc driver test application on GitHub for modification.

The Kernel-3.10 UVC Gadget driver supports two transfer types: Isochronous Transfer and Bulk Transfer. It uses isochronous transfer by default, with a default transfer of 1KB per micro-frame (125 microseconds).

Therefore, the Kernel-3.10 UVC Gadget can have 4 different transfer configuration methods. Different configuration methods result in different maximum USB bandwidth and frame rates.

**UVC Gadget Transfer Configuration Method 1:**

- Isochronous transfer, 1KB per micro-frame (125 microseconds) (default configuration)
- USB maximum supported bandwidth: **8000 KBps, i.e., 7.8 MBps**
- Maximum supported frame rates are shown in Table 3-3:

Table 3-3 RV1108 USB Gadget Webcam Maximum Frame Rate [Configuration 1]

| USB Version | YUYV Format | MJPG Format | H264 Format |
| --- | --- | --- | --- |
| USB 2.0 | 640 × 480 @12fps | 1920 × 1080 @30fps | 1920 × 1080 @30fps |

**UVC Gadget Transfer Configuration Method 2:**

- Isochronous transfer, 2KB per micro-frame (125 microseconds).

  **Note:** The following patch needs to be updated. At the same time, note that increasing the transfer bandwidth may cause screen flickering. If this occurs, consider increasing the USB QOS priority, increasing the USB AHB CLK (aclk_peri and hclk_peri) to above 150MHz, and other methods to improve USB DMA transfer efficiency.

```
diff --git a/drivers/usb/gadget/f_uvc.c b/drivers/usb/gadget/f_uvc.c
index 382536d..17eac12 100644
--- a/drivers/usb/gadget/f_uvc.c
+++ b/drivers/usb/gadget/f_uvc.c
@@ -37,7 +37,7 @@ static unsigned int streaming_interval = 1;
 module_param(streaming_interval, uint, S_IRUGO|S_IWUSR);
 MODULE_PARM_DESC(streaming_interval, "1 - 16");

-static unsigned int streaming_maxpacket = 1024;
+static unsigned int streaming_maxpacket = 2048;
```

- USB maximum supported bandwidth: **16000 KBps, i.e., 15.62 MBps**
- Maximum supported frame rates are shown in Table 3-4:

Table 3-4 RV1108 USB Gadget Webcam Maximum Frame Rate [Configuration 2]

| USB Version | YUYV Format | MJPG Format | H264 Format |
| --- | --- | --- | --- |
| USB 2.0 | 640 × 480 @24fps | 1920 × 1080 @30fps | 1920 × 1080 @30fps |

**UVC Gadget Transfer Configuration Method 3:**

- Isochronous transfer, 3KB per micro-frame (125 microseconds)

  **Note:** The following patch needs to be updated. At the same time, note that increasing the transfer bandwidth may cause screen flickering. If this occurs, consider increasing the USB QOS priority, increasing the USB AHB CLK (aclk_peri and hclk_peri) to above 150MHz, and other methods to improve USB DMA transfer efficiency.

```
diff --git a/drivers/usb/gadget/f_uvc.c b/drivers/usb/gadget/f_uvc.c
index 382536d..17eac12 100644
--- a/drivers/usb/gadget/f_uvc.c
+++ b/drivers/usb/gadget/f_uvc.c
@@ -37,7 +37,7 @@ static unsigned int streaming_interval = 1;
 module_param(streaming_interval, uint, S_IRUGO|S_IWUSR);
 MODULE_PARM_DESC(streaming_interval, "1 - 16");

-static unsigned int streaming_maxpacket = 1024;
+static unsigned int streaming_maxpacket = 3072;
```

- USB maximum supported bandwidth: **24000 KBps, i.e., 23.44 MBps**
- Maximum supported frame rates are shown in Table 3-5:

Table 3-5 RV1108 USB Gadget Webcam Maximum Frame Rate [Configuration 3]

| USB Version | YUYV Format | MJPG Format | H264 Format |
| --- | --- | --- | --- |
| USB 2.0 | 640 × 480 @30fps | 1920 × 1080 @30fps | 1920 × 1080 @30fps |

**UVC Gadget Transfer Configuration Method 4:**

- Bulk transfer

> **Note:** The following patch needs to be updated. In addition, bulk transfer does not support resolution switching. If multiple resolutions need to be supported, bulk transfer is not recommended.

```
diff --git a/drivers/usb/gadget/f_uvc.c b/drivers/usb/gadget/f_uvc.c
index d146ce7..9eb7aad 100644
--- a/drivers/usb/gadget/f_uvc.c
+++ b/drivers/usb/gadget/f_uvc.c
@@ -45,7 +45,7 @@ static unsigned int streaming_maxburst;
 module_param(streaming_maxburst, uint, S_IRUGO|S_IWUSR);
 MODULE_PARM_DESC(streaming_maxburst, "0 - 15 (ss only)");

-static bool bulk_streaming_ep;
+static bool bulk_streaming_ep = true;
 module_param(bulk_streaming_ep, bool, S_IRUGO | S_IWUSR);
```

- USB maximum supported bandwidth: **~35 MBps**
- Maximum supported frame rates are shown in Table 3-6:

Table 3-6 RV1108 USB Gadget Webcam Maximum Frame Rate [Configuration 4]

| USB Version | YUYV Format | MJPG Format | H264 Format |
|---|---|---|---|
| USB 2.0 | 640 × 480 @30+fps | 1920 × 1080 @30+fps | 1920 × 1080 @30+fps |

### 3.3.2 USB Gadget Webcam [Configfs-UVC]

Linux-4.4 and later kernel versions support configuring UVC functionality through the configfs framework. Some chip platforms such as RV1126B, RK3588, and RK3576 can support USB 3.0 UVC functionality.

Table 3-7 Rockchip Platform UVC Support

| USB Version | UVC 1.1 | UVC 1.5 | NV12 | YUYV | MJPG | H264 | H265 |
|---|---|---|---|---|---|---|---|
| USB 2.0 | ✅ | ❌ | ✅ | ✅ | ✅ | ✅ | ✅ |
| USB 3.0 | ✅ | ❌ | ✅ | ✅ | ✅ | ✅ | ✅ |

Table 3-8 UVC configfs Parameters and Their Functions

| Parameter Name | Configurable Values | Applicable Scope | Description |
| --- | --- | --- | --- |
| streaming_maxpacket | 1024/2048/3072 (unit: byte) | USB 2.0/3.0 ISOC transfer | Sets the maximum data packet transferred per micro-frame for USB 2.0<br>Sets the maximum data packet supported per burst transfer for USB 3.0 |
| streaming_maxburst | 0 ~ 15 | USB 3.0 ISOC/BULK transfer | Sets the maximum number of bursts supported per USB transfer transaction.<br>When calculating bandwidth, use streaming_maxburst + 1 |
| streaming_interval | 1/2/3/4 | USB 2.0/3.0 ISOC transfer | Sets the ISOC video stream transfer interval<br>time = 2^(streaming_interval-1) x 125 μs |
| streaming_bulk | 0/1 | USB 2.0/3.0 BULK transfer | 0: Use ISOC to transfer video stream<br>1: Use BULK to transfer video stream |
| streaming_bulk_hs_maxpayload | 131072 ~ 1048576 (unit: byte) | USB 2.0 BULK transfer | Sets the maxpayload size for USB 2.0 BULK transfer<br>The recommended configuration value is a multiple of 131072 (128KB) |

**UVC Transfer Rate and Hardware Limitations**

1. UVC BULK Mode is Best-Effort, with no reserved bandwidth, transmitting only when the bus is idle, and no latency guarantee. In scenarios where the USB bus bandwidth is exclusively used by UVC, the maximum rate of BULK transfer is better than ISOC transfer;
2. USB 2.0 ISOC transfer rate calculation formula = streaming_maxpacket × 8000 × 8 / 2^(streaming_interval-1) / 1024 Kbps;
3. USB 3.0 ISOC transfer rate calculation formula = streaming_maxpacket × (streaming_maxburst + 1) × 8000 × 8 / 2^(streaming_interval-1) / 1024 Kbps;
4. streaming_maxburst and streaming_maxpacket are not recommended to be set too large. Because the larger the value, the larger the corresponding USB controller endpoint hardware TxFIFO needs to be. However, the total size of the actual hardware TxFIFO is limited (the total TxFIFO size of the DWC3 controller is 16KB). In UVC composite device scenarios, FIFO shortage problems are likely to occur.

Table 3-9 Rockchip Platform USB 3.0 UVC Measured Maximum Transfer Rate

| Transfer Type | Maximum Rate | Test Condition Description |
| --- | --- | --- |
| USB 3.0<br>UVC ISOC | NV12 format 3840 x 2160@23fps | Based on RK3576 EVB1<br>Configure streaming_maxburst = 11, streaming_maxpacket = 3072;<br>Configure dwc3 dts attribute tx-fifo-resize;<br>Modify dwc3 driver __dwc3_gadget_resize_tx_fifos() to increase the mult of isoc ep to 10 |
| USB 3.0<br>UVC BULK | NV12 format 3840 x 2160@30fps | Based on RV1126B EVB1<br>Configure streaming_maxburst = 5, streaming_maxpacket = 1024;<br>Configure dwc3 dts attribute tx-fifo-resize; |

## 3.4 USB Rndis Transfer Rate Analysis

USB Rndis is essentially TCP/IP over USB, which runs TCP/IP on USB devices, making the USB device look like a network adapter.

**Test Method:**

Test tool: **iperf**

For detailed description of the iperf tool, please refer to USB Ethernet Transfer Rate Analysis

**USB Rndis kernel config settings are as follows:**

- Linux-3.10 Android Gadget USB Rndis configuration method:

```
Device Drivers  --->
   [*] USB support  --->
       <*>   USB Gadget Support  --->
           <*>   USB Gadget Drivers (Android Composite Gadget)  --->
                 Android Composite Gadget
```

Corresponding device node:

Under the /sys/class/android_usb/android0/f_rndis directory, there are some nodes that can be used to configure the network adapter method and commands

- Linux-3.10 Gadget USB Rndis configuration method:

```
Device Drivers  --->
   [*] USB support  --->
       <*>   USB Gadget Support  --->
           <*>   USB Gadget Drivers (Ethernet Gadget (with CDC Ethernet support))  --->
                 Ethernet Gadget (with CDC Ethernet support)
```

- Linux-4.4 Configfs USB Rndis configuration method:

```
Device Drivers  --->
   [*] USB support  --->
       <*>   USB Gadget Support  --->
           <*>   USB Gadget Drivers (USB functions configurable through configfs)  --->
                 [*]       RNDIS
```

**Example:**

Using Linux-3.10 Android Gadget USB Rndis as the analysis example

The USB Rndis enumeration log is as follows:

```
shell@android:/ # setprop sys.usb.config rndis
[ 2445.809337] rndis_function_bind_config MAC: 36:C8:78:69:C5:D3
[ 2445.809532] android_usb gadget: using random self ethernet address
[ 2445.809666] android_usb gadget: using random host ethernet address
[ 2445.812255] rndis0: MAC 0a:d4:f2:e3:c5:c7
[ 2445.812330] rndis0: HOST MAC ea:88:5c:93:9d:b5
[ 2446.208233] DWC_OTG: ********soft connect!!!****************************************
[ 2446.541253] android_usb gadget: high speed config #1: android
[ 2446.542681] android_work: sent uevent USB_STATE=CONFIGURED
```

**Note:** In the above log, the HOST MAC of rndis0 corresponds to the Ethernet adapter MAC address enumerated on the PC side (such as Windows).

Set the IP and start rndis:

```
ifconfig rndis0 192.168.1.2 up
```

Where the IP address 192.168.1.2 must be in the same network segment as the PC's IP, i.e., both in the 192.168.XX.XX network segment.

For more detailed configuration and test methods, please refer to USB Ethernet Transfer Rate Analysis

**Note**: Before testing, the Windows firewall needs to be disabled, otherwise the Rockchip platform may not be able to ping Windows.

**Test Results:**

The following are all High-speed test results.

Linux-3.10 Android Gadget USB Rndis: **Uplink 20 Mbps, Downlink 40 Mbps**

Linux-3.10 Ethernet Gadget USB Rndis: **Both uplink and downlink 100 Mbps**

Linux-4.4 Configfs USB Rndis: **Both uplink and downlink 100 Mbps**

**Test Result Analysis and Optimization Directions:**

Currently, a known issue is that the uplink and downlink rates of Linux-3.10 Android Gadget USB Rndis are both relatively low and require further optimization.

## 3.5 USB Gadget HID Transfer Rate Analysis

**Test Method:**

Test tools:

PC Windows: HidTest.exe, PortHelper1.7.exe

PC Ubuntu: hid_gadget_test test script from Linux-3.10 Documentation/usb/gadget_hid.txt

Or download HID API

### 3.5.1 USB Gadget HID [Legacy]

**Test Results:**

Testing rk3188 Gadget HID transfer rate, results are as follows:

case1. Set report_length to 512 bytes, interval to 1ms, transfer file size 10768KB, HID is a high speed device

PC -> 3188 Rate: 249 KBps 3188 -> PC Rate: 125 KBps

case2. Set report_length to 512 bytes, interval to 125us, transfer file size 10768KB, HID is a high speed device

PC -> 3188 Rate: 872 KBps 3188 -> PC Rate: 2659 KBps

**Test Result Analysis:**

The transfer rate of USB Gadget HID is mainly determined by the **report_length** of the HID descriptor and the **interrupt transfer interval**. In the Linux USB Gadget HID driver source file (drivers/usb/gadget/hid.c), no HID descriptor is provided. Developers need to refer to the Linux documentation Documentation/usb/gadget_hid.txt for development. The following is reference code for the HID descriptor with report_length set to 512 bytes.

```
index 79afa82..a05ad79 100644
--- a/drivers/usb/gadget/hid.c
+++ b/drivers/usb/gadget/hid.c
@@ -271,10 +272,93 @@ MODULE_DESCRIPTION(DRIVER_DESC);
 MODULE_AUTHOR("Fabien Chouteau, Peter Korsgaard");
 MODULE_LICENSE("GPL");

+static struct hidg_func_descriptor rk_hidg_desc = {
+       .subclass               = 0, /* No subclass */
+       .protocol               = 2, /* Keyboard */
+       .report_length          = 512,
+       .report_desc_length     = 38,
+       .report_desc            = {
+               0x06, 0x00, 0xFF,        /*USAGE_PAGE (Vendor Defined Page 1)*/
+               0x09, 0x01,             /*USAGE (Vendor Usage 1)*/
+               0xA1, 0x01,             /*COLLECTION (Application)*/
+               0x09, 0x02,             /*  USAGE (Vendor Usage 2)*/
+               0x09, 0x03,             /*USAGE (Vendor Usage 3)*/
+               0x15, 0x00,             /*LOGICAL_MINIMUM (0)*/
+               0x26, 0xFF, 0x00, /*LOGICAL_MAXIMUM (255)*/
+               0x75, 0x10,             /*REPORT_SIZE (16)*/
+               0x96, 0x00, 0x01, /* REPORT_COUNT (256) */
```

```
+               0x81, 0x02,             /*INPUT (Data,Var,Abs)  */
+               0x09, 0x04,             /*USAGE (Vendor Usage 4)*/
+               0x15, 0x00,             /*LOGICAL_MINIMUM (0)   */
+               0x26, 0xFF, 0x00, /*LOGICAL_MAXIMUM (255)       */
+               0x75, 0x10,             /*REPORT_SIZE (16)*/
+               0x96, 0x00, 0x01,       /*REPORT_COUNT (256)*/
+               0x91, 0x02,             /*OUTPUT (Data,Var,Abs) */
+               0xC0                    /*END_COLLECTION*/
+       }
+};
```

To support a report_length of 1024 bytes, you can change it to

```
@@ -275,7 +275,7 @@ MODULE_LICENSE("GPL");
 static struct hidg_func_descriptor rk_hidg_desc = {
        .subclass               = 0, /* No subclass */
        .protocol               = 2, /* Keyboard */
-       .report_length          = 512,
+       .report_length          = 1024,
        .report_desc_length     = 38,
        .report_desc            = {
                0x06, 0x00, 0xFF,        /*USAGE_PAGE (Vendor Defined Page 1)*/
@@ -285,13 +285,13 @@ static struct hidg_func_descriptor rk_hidg_desc = {
                0x09, 0x03,             /*USAGE (Vendor Usage 3)*/
                0x15, 0x00,             /*LOGICAL_MINIMUM (0)*/
                0x26, 0xFF, 0x00, /*LOGICAL_MAXIMUM (255)*/
-               0x75, 0x10,             /*REPORT_SIZE (16)*/
+               0x75, 0x20,             /*REPORT_SIZE (32)*/
                0x96, 0x00, 0x01, /* REPORT_COUNT (256) */
                0x81, 0x02,             /*INPUT (Data,Var,Abs)  */
                0x09, 0x04,             /*USAGE (Vendor Usage 4)*/
                0x15, 0x00,             /*LOGICAL_MINIMUM (0)   */
                0x26, 0xFF, 0x00, /*LOGICAL_MAXIMUM (255)       */
-               0x75, 0x10,             /*REPORT_SIZE (16)*/
+               0x75, 0x20,             /*REPORT_SIZE (32)*/
                0x96, 0x00, 0x01,       /*REPORT_COUNT (256)*/
                0x91, 0x02,             /*OUTPUT (Data,Var,Abs) */
                0xC0                    /*END_COLLECTION*/
```

The HID interrupt transfer interval can be modified by changing the bInterval member of the hidg_hs_in_ep_desc and hidg_hs_out_ep_desc structures in the drivers/usb/gadget/f_hid.c file. The native value is 4 (1ms), and the minimum can be changed to 1 (125us).

### 3.5.2 USB Gadget HID [Configfs-HID]

Taking the Linux SDK USB Gadget configfs HID configuration as an example.

1. Configfs-HID Configuration Method

   Kernel configuration: `CONFIG_USB_CONFIGFS_F_HID=y`

   S50usbdevice script to configure custom hid (input output 1024 byte report length):

```
if [ $HID_EN = on ]; then
echo 0 > /sys/kernel/config/usb_gadget/rockchip/functions/hid.usb0/protocol
echo 0 > /sys/kernel/config/usb_gadget/rockchip/functions/hid.usb0/subclass
echo 1024 > /sys/kernel/config/usb_gadget/rockchip/functions/hid.usb0/report_length
echo -ne
\\x05\\x0c\\x09\\x01\\xA1\\x01\\x09\\x01\\x15\\x00\\x26\\xFF\\x00\\x75\\x10\\x96\\x00\\x0
2\\x81\\x02\\x09\\x01\\x15\\x00\\x26\\xFF\\x00\\x75\\x10\\x96\\x00\\x02\\x91\\x02\\xc0 >
/sys/kernel/config/usb_gadget/rockchip/functions/hid.usb0/report_desc
syslink_function hid.usb0
fi
```

2. Configfs-HID Enable

```
echo usb_hid_en > /tmp/.usb_config
/etc/init.d/S50usbdevice restart
```

3. Configfs-HID Typical Issue Handling

   For Linux-5.10 and earlier kernel versions, when the USB DWC3 controller driver enables the tx-fifo-resize function (typically used in UVC composite device scenarios), it limits HID to a maximum of 64 bytes per data packet transfer. To support data packet transfers larger than 64 bytes, the following patch needs to be updated.

```
diff --git a/drivers/usb/dwc3/gadget.c b/drivers/usb/dwc3/gadget.c
index 05d75cb4ab64..8e7890fee360 100644
--- a/drivers/usb/dwc3/gadget.c
+++ b/drivers/usb/dwc3/gadget.c
@@ -845,11 +845,13 @@ static int __dwc3_gadget_resize_tx_fifos(struct dwc3_ep *dep)
                }
        } else if (usb_endpoint_xfer_int(dep->endpoint.desc)) {
                /*
-                * REVISIT: we assume that the maxpacket of interrupt
-                * endpoint is 64 Bytes for MTP and the other functions.
+                * Set one maxpacket size for Interrupt endpoints,
+                * and at least 64 Bytes for MTP functions.
                 */
                mult = 1;
-               maxpacket = 64;
+               maxpacket = dep->endpoint.maxpacket;
+               if (maxpacket < 64)
+                       maxpacket = 64;
        } else {
                goto out;
```